

Breakout Week 1 — Group 4b

Ride-Hailing Trips — Completed Worksheet

Run together in DBeaver against `raw_rides.trips` and `raw_rides.zones` (mirrored here from `data/raw/trips.csv` and `data/raw/zones.csv` — same four-table structure, same column names). All six queries below were run as a group; answers are filled in below each one.

Part A — Six Questions

1. How big is your data?

```
SELECT count(*) FROM raw_rides.trips;
```

ANSWER

28,112 rows. The simplest query there is — and it ran clean.

2. What period does it cover?

```
SELECT min(trip_date) AS earliest,  
       max(trip_date) AS latest  
FROM raw_rides.trips;
```

ANSWER

earliest = **01-Apr-2024** latest = **31/12/2024** — looks like a one-year window. It isn't. See Part B.

3. How many different values does status have?

```
SELECT status, count(*) AS rows  
FROM raw_rides.trips  
GROUP BY status  
ORDER BY status;
```

ANSWER

12 distinct values came back, not 3. Same three real statuses spelled 12 ways — case and stray whitespace: “Completed” / “COMPLETED” / “completed” / “ Completed ” (21,145 / 1,041 / 961 / 974 rows), same four-way split for both cancellation reasons.

4. How much is missing?

```
SELECT count(*) AS total,  
       count(fare) AS have_a_value,  
       count(*) - count(fare) AS missing
```

```
FROM raw_rides.trips;
```

ANSWER

total 28,112 · have a value 26,706 · missing 1,406.

5. Is every row unique?

```
SELECT count(*) AS rows,  
       count(DISTINCT trip_id) AS unique_ids,  
       count(*) - count(DISTINCT trip_id) AS extras  
FROM raw_rides.trips;
```

ANSWER

28,112 rows · 28,000 unique IDs · 112 extras. Checked by hand afterward: every one of the 112 is an exact full-row duplicate, not just a repeated ID with different data.

6. STRETCH — which pickup zones are busiest?

```
SELECT zones.zone_name,  
       count(*) AS rows  
FROM raw_rides.trips  
JOIN raw_rides.zones ON trips.pickup_zone_id = zones.zone_id  
GROUP BY zones.zone_name  
ORDER BY rows DESC  
LIMIT 5;
```

zone_name	rows
Teshie	2,007
Lapaz	1,930
Weija	1,904
Madina	1,900
Airport City	1,891

Part B — Talk About It

Which of those six answers surprised you most, and why?

ANSWER

Q3 — 12 statuses instead of 3. A duplicate ID or a missing value is a number telling you something's wrong, but a GROUP BY silently multiplying categories by 4× doesn't throw an error and looks like a perfectly normal result table. The only way to catch it is to actually count the rows the instructions told you to count.

Question 2 gave a nonsense date range. Work out why. What does that tell you about the column?

ANSWER

MIN/MAX on trip_date is comparing strings, not dates — the column mixes at least four formats (2024-06-01, 2024/12/06, 14-Dec-2025, 29/05/2025). “01-Apr-2024” sorts lowest alphabetically because “0” is a low character, not because April 2024 is early. That tells us trip_date is stored as text, not a real date type, and nothing built on it — filtering, sorting, date math — can be trusted until it's parsed properly.

If you built a dashboard on this data today, without fixing anything, name one number that would be wrong.

ANSWER

Any revenue total. Summing raw fare doesn't catch the 112 duplicate rows — every duplicated trip's fare gets counted twice, so total revenue is overstated by whatever those 112 fares sum to. Trip-count KPIs have the identical problem for the identical reason.

Your question asks about fare. Which of your four tables do you actually need? Do you need all four?

ANSWER

Just trips — fare lives there, one row per ride, no join required. zones, drivers, or riders only come in the moment you want to break fare down by something (by zone, by driver, by payment method). For a bare total or average fare, pulling in a fourth table only adds a chance of a bad join, not accuracy.

Name one thing you now know you will have to fix before week 4.

ANSWER

The status column, specifically — because it's the one that fails silently. A stakeholder-facing chart grouped by raw status shows up to 12 slices where there should be 3, and nobody reviewing that chart would know to question it unless they'd already run Q3 themselves.

Part C — Report Back to the Class

Ninety seconds. Two things only.

One number that surprised us:

ANSWER

Grouping trips by raw status returns 12 rows, not 3 — the same three outcomes spelled with different case and stray whitespace, so an un-normalized GROUP BY silently fragments every downstream count.

One question:

ANSWER

Given trip_date is stored as text and mixes at least four different formats, do we standardize it in SQL at read time on every query, or fix it once in a cleaning step and load a corrected date column back — because every query on this sheet ran fine against the raw text, and that's exactly the problem.